

HW 4

Sam Ly

February 24, 2026

Final code:

<https://github.com/samlyme/Assignments/tree/main>

Chapter 8

The code for the challenge questions can be found here:

<https://github.com/samlyme/Assignments/tree/cs4080/ch8/challenge>

1. The REPL no longer supports entering a single expression and automatically printing its result value. That's a drag. Add support to the REPL to let users type in both statements and expressions. If they enter a statement, execute it. If they enter an expression, evaluate it and display the result value.

This can be done by performing a simple check for semicolons in the provided token list. If there are no semicolons, then we treat it as an expression.

```
private static void run(String source) {
    Scanner scanner = new Scanner(source);
    List<Token> tokens = scanner.scanTokens();
    Parser parser = new Parser(tokens);

    boolean hasSemicolon = false;
    for (Token token : tokens) {
        if (token.type == TokenType.SEMICOLON) {
            hasSemicolon = true;
            break;
        }
    }

    if (hasSemicolon) {
        List<Stmt> statements = parser.parse();

        if (hadError) return;

        interpreter.interpret(statements);
    } else {
        Expr expr = parser.parseExpression();
        System.out.println(interpreter.evaluate(expr));
    }
}
```

2. Maybe you want Lox to be a little more explicit about variable initialization. Instead of implicitly initializing variables to nil, make it a runtime error to access a variable that has not been initialized or assigned to, as in:

This can be done by adding a HashSet that tracks which variables have been initialized.

```

package com.mylox.lox;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

class Environment {
    final Environment enclosing;
    private final Map<String, Object> values = new HashMap<>();
    private final Set<String> initialized = new HashSet<>();

    Environment() {
        this.enclosing = null;
    }

    Environment(Environment enclosing) {
        this.enclosing = enclosing;
    }

    void define(String name, Object value) {
        values.put(name, value);
        if (value != null) initialized.add(name);
    }

    Object get(Token name) {
        if (values.containsKey(name.lexeme)) {
            if (!initialized.contains(name.lexeme)) {
                throw new RuntimeError(name, "Variable not initialized.");
            }
            return values.get(name.lexeme);
        }

        if (enclosing != null) return enclosing.get(name);

        throw new RuntimeError(name,
            "Undefined variable '" + name.lexeme + "'");
    }

    void assign(Token name, Object value) {
        if (values.containsKey(name.lexeme)) {
            values.put(name.lexeme, value);
            initialized.add(name.lexeme);
            return;
        }

        if (enclosing != null) {
            enclosing.assign(name, value);
            return;
        }

        throw new RuntimeError(name, "Undefined variable '" + name.lexeme + "'");
    }
}

```

3. What does the following program do?

```
var a = 1;
{
  var a = a + 2;
  print a;
}
```

What did you expect it to do? Is it what you think it should do? What does analogous code in other languages you are familiar with do? What do you think users will expect this to do?

This creates a block scope that redeclares the variable `a` as 3. When the block scope exits, `a` is still 1. This behaves how I would expect it to. This behavior is also useful as it lets the user redefine variables with the same name in scopes without causing strange mutations outside. This is analogous to how JS handles scopes.